Exercise 2 - ROS Development & Kinematics

Sami Jagirdar [ccid: jagirdar] | Basia Ofovwe [ccid: ofovwe]

Technologies used: Python, ROS, OpenCV

Part 1. ROS Basics

1.1. ROS Core Concepts

Node

A ROS node is an independent process that performs a specific task, such as reading sensors or controlling motors.

Nodes communicate via topics, services, and actions, enabling modular system design.

Example: A camera node captures images, while another node processes the image data.

Topic

A ROS topic is a communication channel where nodes exchange messages asynchronously in a publish-subscribe manner.

Used for continuous data streams like sensor readings and control commands.

Example: A camera node publishes images to the `/camera/image_raw` topic, and an analysis node subscribes.

Service

A ROS service enables synchronous request-response communication between nodes.

Used for on-demand operations like retrieving system status or resetting sensors.

Print

Example: `/get_temperature` service returns the current temperature.

Message

A ROS message is a structured data format used for exchanging information between nodes.

Messages are defined in `.msg` files and contain headers and fields like numbers, strings, or nested structures.

Example: The `/wheels_driver_node/wheels_cmd` topic uses `duckietown_msgs.WheelsCmdStamped` type that has `vel_left` & `vel_right` fields to control the throttle of each wheel.

Bag

A ROS bag is a file format for recording and replaying ROS messages, useful for debugging and analysis.

Recorded data can be replayed or plotted to analyze the expected behaviour of the robot

Example: `rosbag record` is used to capture robot's sensor readings are recorded in a `.bag` file for offline debugging. `rosbag play` can be used to play the recorded data to simulate real time operation

1.2. Using ROS with Duckiebots

Duckiebots are ROS-based robots that use ROS nodes to control their movement, sensors, and other functionalities. The Duckiebot software stack is designed to work with ROS, making it easy to develop and test algorithms for autonomous driving.

To use ROS with Duckiebots we setup the DTROS interface using the DTProject template repository that also allows us to create catkin packages that can be built into docker images and ran in containers on the duckiebot

We then wrote our first publisher and subscriber nodes. The publisher node sends a "Hello..." message to a custom topic called `/chatter`

The subscriber node subscribes to the aforementioned topic and reads the data being published

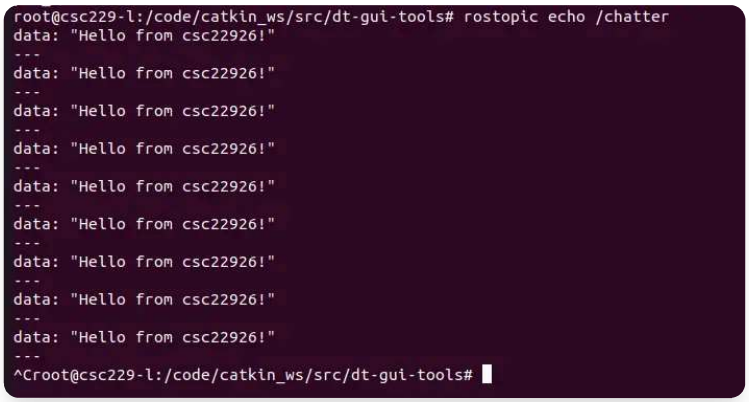
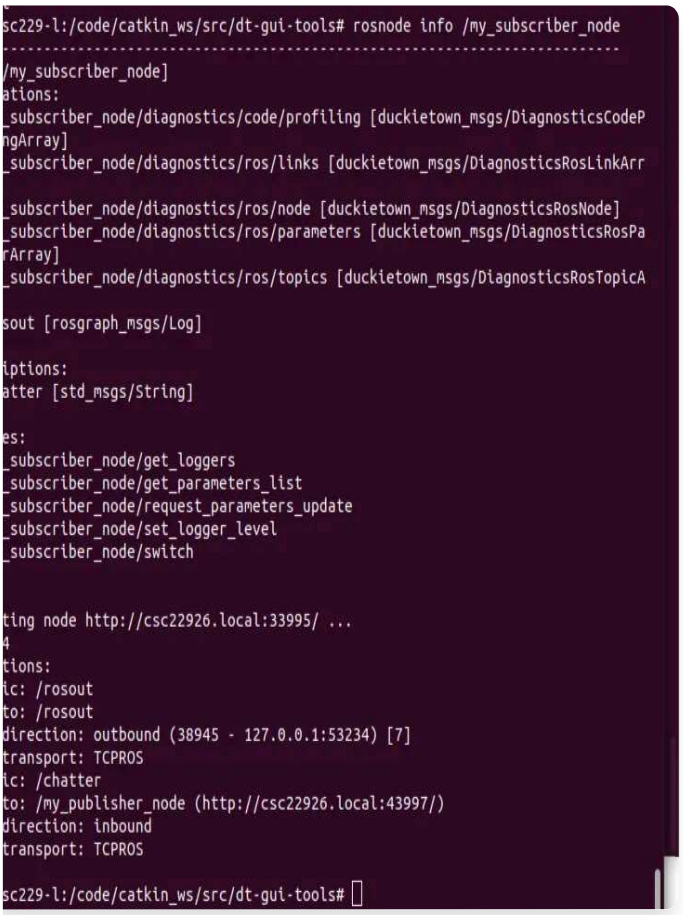


Fig 1.1 Custom ROS Topic `/chatter` Info. The publisher node sends `'Hello from ROBOTNAME'` messages to this topic



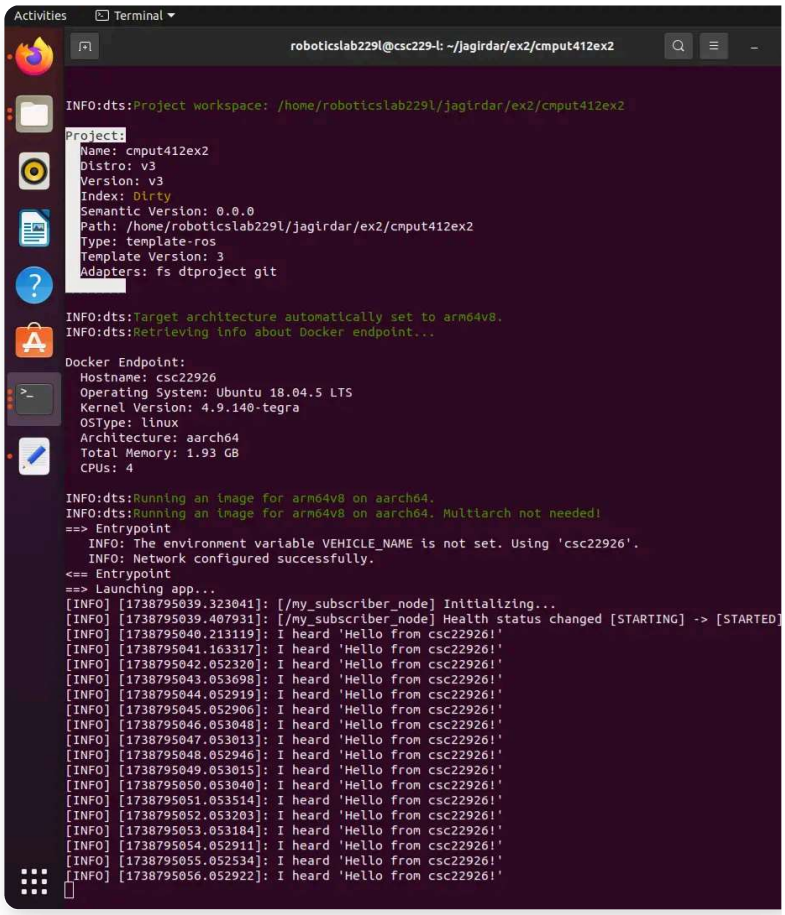


Fig 1.2 Subscriber Node Info showing it has subscribed to the `/chatter` topic. It also shows that the publisher node is connected to the topic.
Fig 1.3 Subscriber Node Output showing the messages received from the publisher node

1.3. Using OpenCV in ROS and Basic Image Processing

Next, we created a node that subscribes to the `/csc22926/camera_node/image/compressed` topic. Then using OpenCV, grayscales this image and annotates it.

The modified image is published to a custom topic

`/csc22926/camera_node/annotated_image/compressed`

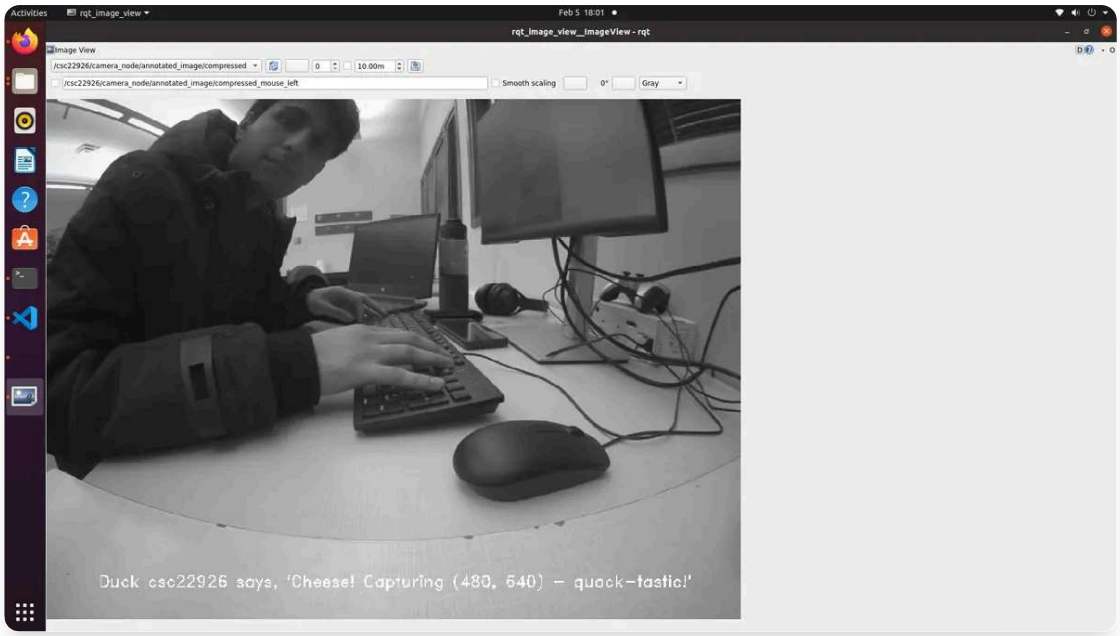


Fig 1.4 Annotated & Grayscaled Image in rqt-viewer which is started using the `rqt_image_view` command. rqt viewer allows use to view `CompressedImage` type data from the `/csc22926/camera_node/annotated_image/compressed` topic. The annotation includes the ROBOT NAME and the size of the image.

Part 2. Odometry Using Wheel Encoders

2.1. Odometry Background

Odometry is the use of data from motion sensors to estimate change in position over time. In differential drive robots like Duckiebots, odometry is calculated using wheel encoders that measure wheel rotation. Particularly, the wheel encoders measure the incremental ticks rotated by each wheel since being turned on. For the DT series bots, the total ticks per revolution, `N_TOTAL_TICKS = 135`. Also, the wheel radius, `WHEEL_RADIUS = 0.0318m` and distance between the center of wheels, `WHEELBASE = 0.09m`
Therefore, we can derive the total distance travelled by each wheel as:
(1) `DISTANCE = (2 * π * WHEEL_RADIUS * ticks) / N_TOTAL_TICKS`

The angle rotated by the bot can be calculated based on the distance travelled by each wheel for some time period
(2) `θ = (ARC_DISTANCE_RIGHT - ARC_DISTANCE_LEFT) / WHEELBASE`
* Arc distance is essentially distance travelled by wheel

We can read the ticks by subscribing to the `/csc22926/left_wheel_encoder_node/tick` and `/csc22926/right_wheel_encoder_node/tick` topics

We can also provide velocity to each of the wheels by publishing the throttle for each wheel to the `/csc22926/wheels_driver_node/wheels_cmd` topic

2.2. Straight Line Task

Using the information above, our first task was to make the duckiebot move 1.25m in a straight line forward and then in reverse along the same path.

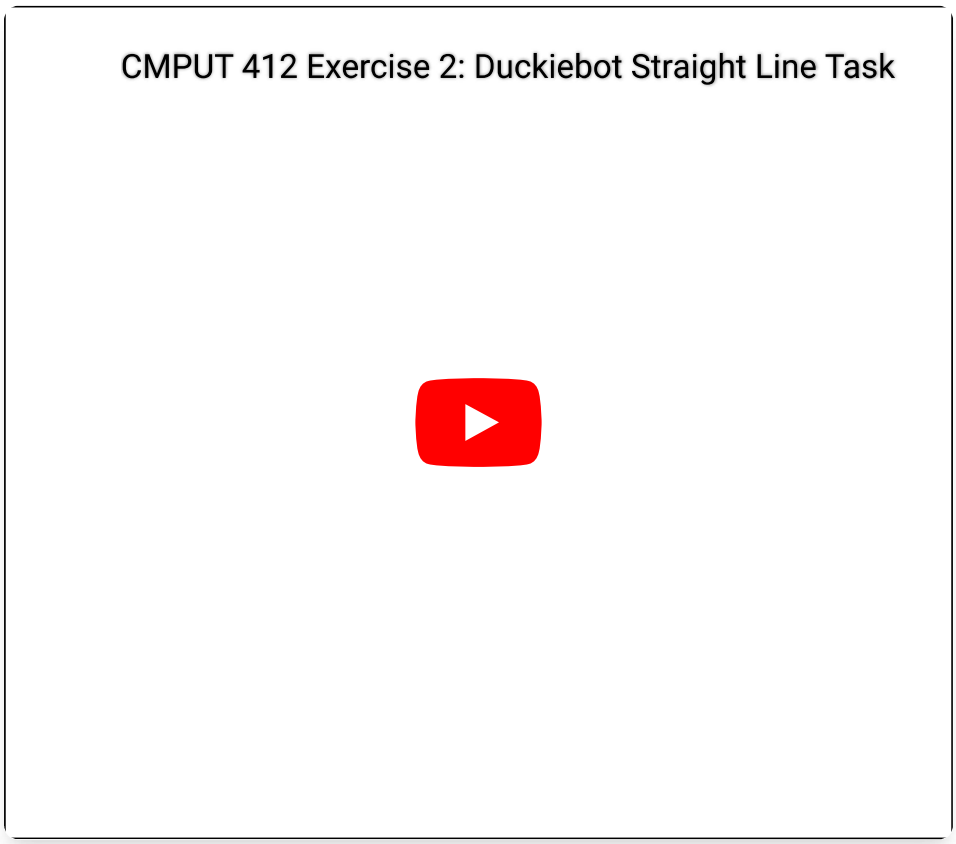


Fig 2.1: csc22926 travels autonomously in a straight line for 1.25m forwards and backwards

Why is there a difference between the actual and desired location?

There are several reasons for the difference between the actual and desired location:

- Wheel slippage: The wheels may slip on the surface, causing the robot to move less than expected. Our equations are based on the constraint that the bot is undergoing perfect rolling, but this is not realistic

- Wheel alignment: The wheels may not be perfectly aligned or incorrectly callibrated (one wheel receives more power than the other), causing the robot to veer off course.
- Wheel Radius: The distance travelled measurement depends on the wheel radius as well which has to be measured manually or via an experiment. An inaccurate measurement can lead to incorrect calculation of actual distance travelled.

Because of these reasons, even though our program detects that the wheels have rotated enough ticks to cover our desired distance, the actual distance covered by the robot can differ. In our case, the bot travelled a distance slightly less than 1.25m as expected.

What speed did we use?

We used a speed of 0.3 (i.e a throttle of 0.3) for both wheels for the forward and reverse motion.

What happens when we increase or decrease the speed?

Increasing the speed would make it more likely for the robot to slip and not cover the full distance, whereas a slower speed may see the robot wheels not having enough torque to rotate on the mat surface as the friction may be too strong. Note that the speed we provide is essentially the throttle, so there are forces and torques involved

2.3. Rotation Task

Next, we made the duckiebot rotate 90 degrees clockwise and then 90 degrees counterclockwise.

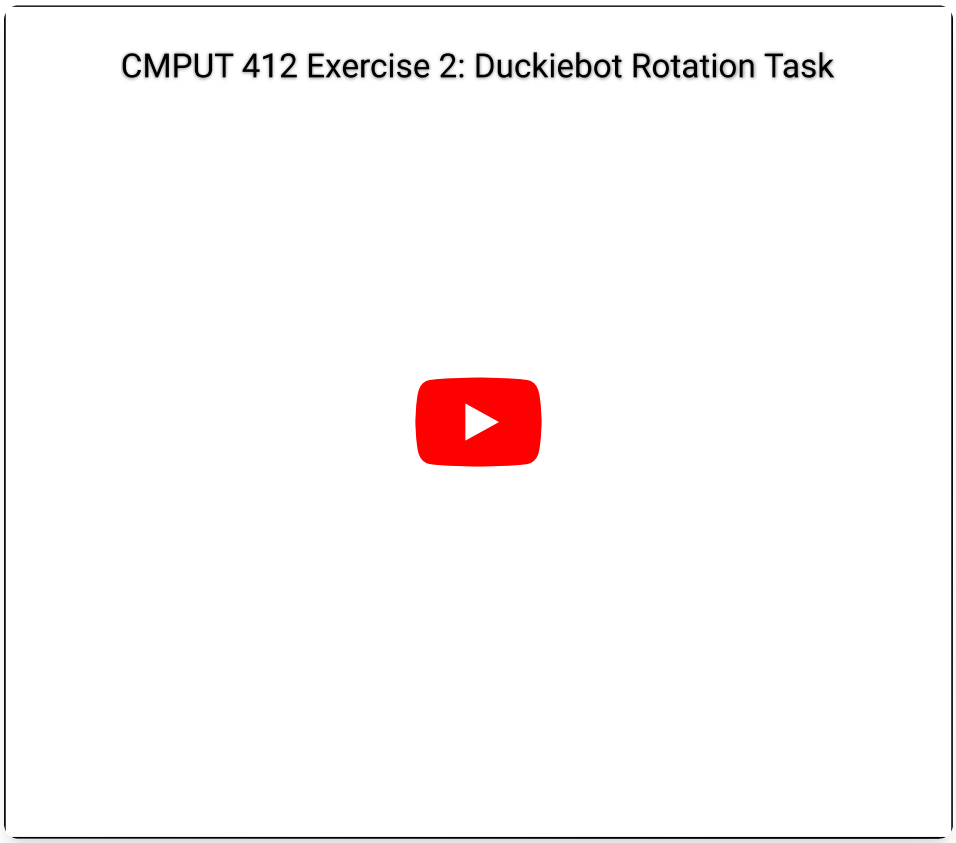


Fig 2.2: csc22926 autonomously rotates 90 degree clockwise and then 90 degree counter-clockwise on the spot

Did we observe any deviations in the rotation?

Yes, we noticed that the bot rotated slightly more than 90 degrees, despite our calculations being ideal for a 90 degree rotation

If deviations exist, what could be the cause

There are several reasons for the deviation:

- Wheel slippage: Similar to the straight line task, the wheels can slip during rotation. Normally, this would make the robot undershoot the 90 degree mark, but we must also consider horizontal sliding which can make it rotate not due to the wheel rotation but because of slipping tangential to the direction of rotation
- The equation for angle rotated depends on the length of the wheelbase of the duckiebot. In our experiment, it was measured manually with a ruler. If the measurement is inaccurate, i.e. greater than the actual length of the wheelbase, then based on equation (2), the wheels would rotate more than required

- The subscriber to the ticks is reading it at a particular rate. It is possible that in its read cycles, it reads the ticks a few moments after the bot has already rotated 90 degrees. So it realizes this and stops the bot from rotating a little too lates

Does our program shutdown after finishing the above tasks?

Yes, our terminates and shuts down after executing the above tasks. This is further evidenced by the fact that running the `docker ps` command and seeing that our launched program does not exist.

Potential reasons for why it may not shutdown is that the ros nodes could still be active after our particular task is ended, especially with ros commands like spin() which keep subscriber nodes alive even after it reaches the end of the python script to keep listening to messages and that the ros shutdown() command isn't called. In our code, we don't have a call to `rospy.spin()` (unless required for the purposes of a subscriber node), we also have a break statement in our while loop whose condition is to check if `rospy.is_shutdown()` is true or not. After exiting the while loop, in our main function, we call `rospy.signal_shutdown()` as well, ensuring that the ROS master terminates all nodes.

2.4. Plotting Trajectory

The velocity (throttle) messages we publish to the `/csc22926/wheels_driver_node/wheels_cmd` topic along with the timestamps at which we publish them can be used to determine the change in x and y position of the robot in the world frame of reference

The velocity information is recorded into a bag file using the `rosbag record topic -o filename.bag` command

We then load the bag file contents in a python script and apply the following kinematics equations to get our x and y positions of the robot in the world frame at each timestamp

(3) $\Delta x_{robot} = (v_{left} + v_{right}) / 2 * \Delta t * SCALING\_FACTOR$

(4) $\Delta y_{robot} = 0$

(5) $\Delta \theta_{robot} = (v_{right} - v_{left}) / WHEELBASE * \Delta t * SCALING\_FACTOR$

(6) $\Delta x_{world} = \Delta x_{robot} * \cos(\theta) - \Delta y_{robot} * \sin(\theta)$

(7) $\Delta y_{world} = \Delta x_{robot} * \sin(\theta) + \Delta y_{robot} * \cos(\theta)$

(8) $x_{world} = x_{world} + \Delta x_{world}$

(9) $y_{world} = y_{world} + \Delta y_{world}$

(10) $\theta = \theta + \Delta\theta_{robot}$

Where $SCALING\_FACTOR = 7.5$ is used to scale the throttle to the actual velocity of the robot and we start with initial values of $(x_{world}, y_{world}, \theta) = (0, 0, \pi/2)$.
Note that the published velocities are not in exact units of m/s like how WHEELBASE and Δt are. These are just very small values [-1,1] that are **proportional** to the actual velocities of the wheels. Therefore, we made an educated guess on the proportionality constant and multiplied the throttles with this constant (SCALING_FACTOR) to get an accurate measurement of $\Delta\theta_{robot}$ and Δx_{robot}

We then plot the trajectory of the robot in the world frame using the matplotlib library

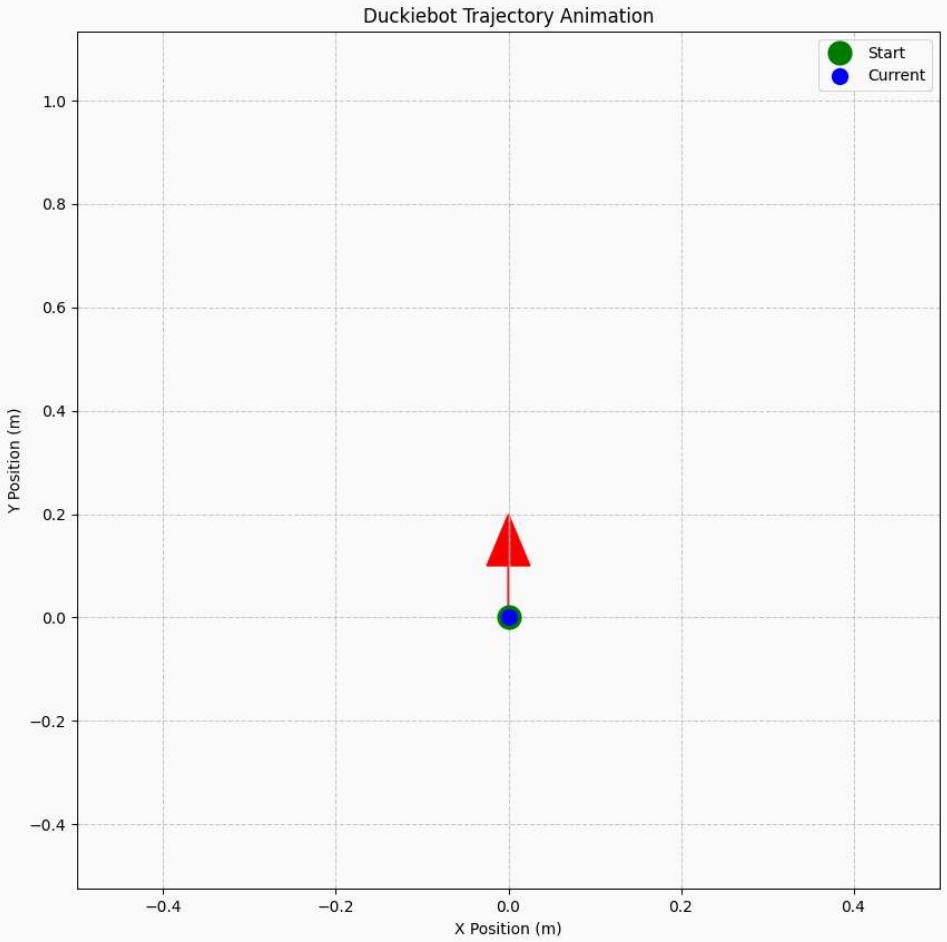


Fig 2.3: Trajectory of the straight line task according to csc22926. (not necessarily the same as actual path travelled)

* the axes say that x and y are in m, this is not necessarily correct and was a typo. For them to be meters, the SCALING_FACTOR would have to be exactly the correct value to convert between throttle and velocity in m/s

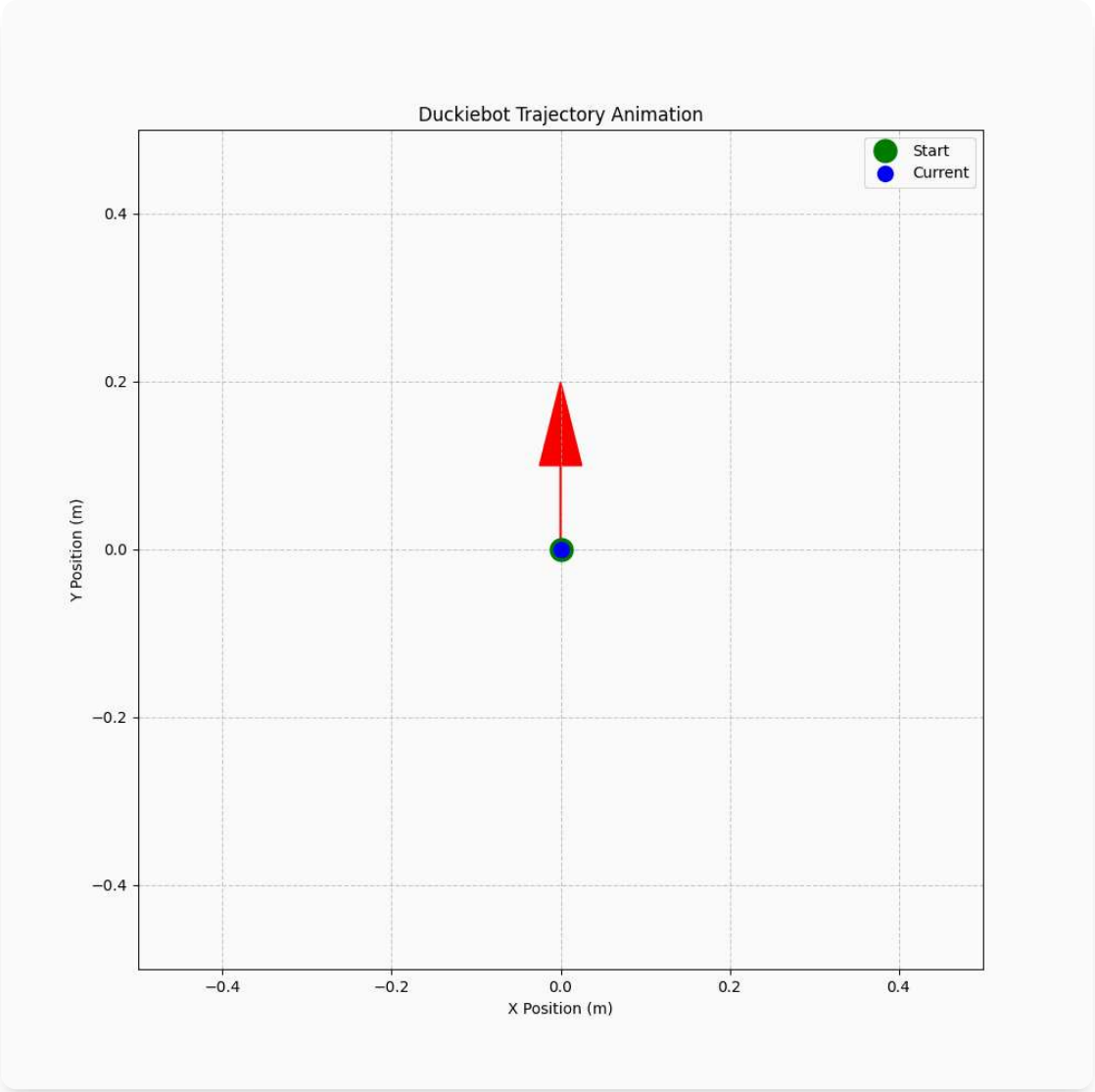


Fig 2.3: Trajectory of the 90 degree rotation task according to csc22926 (not necessarily the same as actual path travelled). As expected, the position of the bot does not change

Part 3. The Will of D

Our final task for this project was to use the wheel encoders for odometry to make the Duckiebot follow 'D'-shaped path. Along the way, we impleted an LED light service to indicate and represent different states of the bots motion

3.1. Node 1: Moving the Duckiebot in D-shaped path

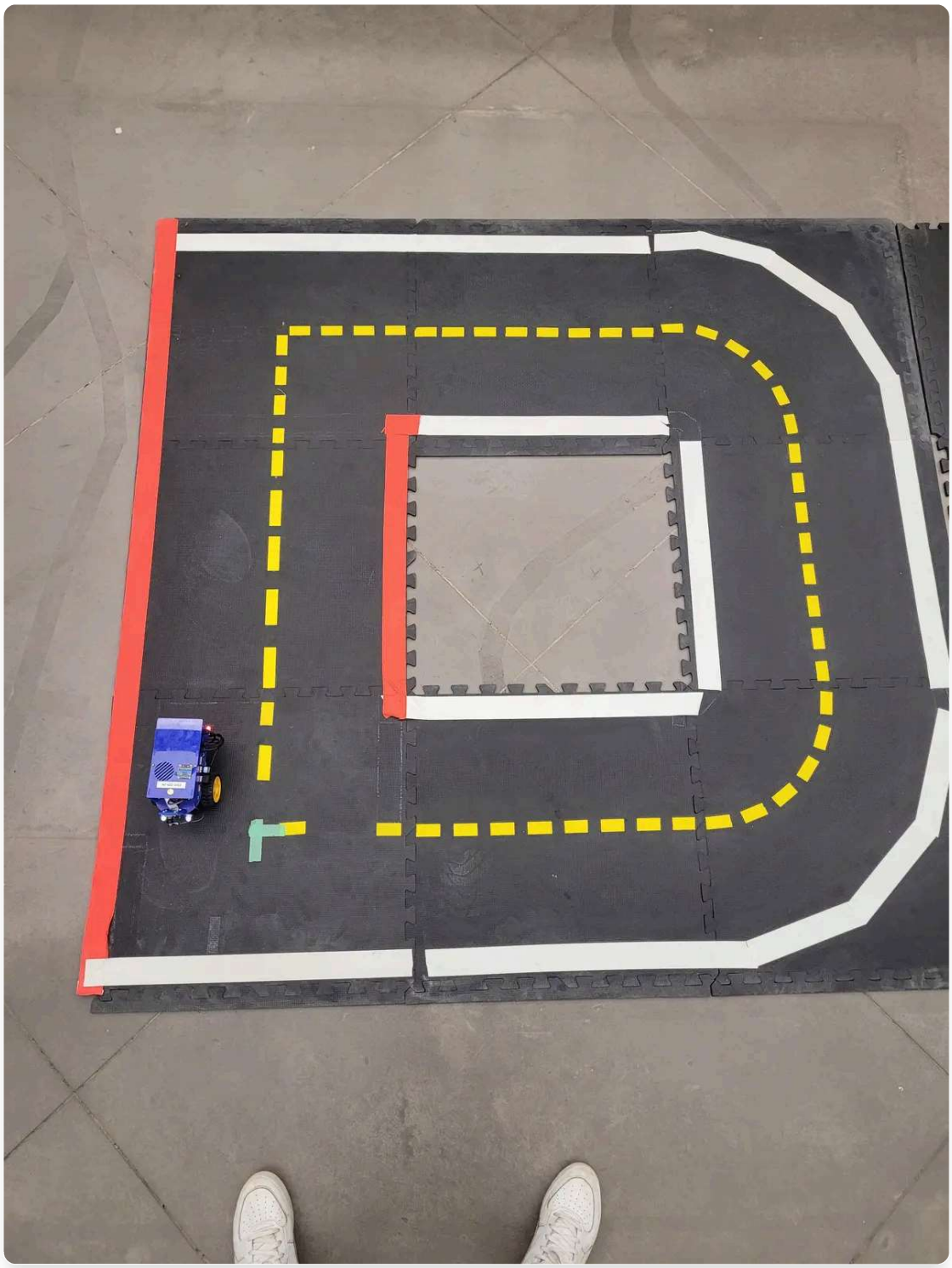


Fig 3.1: The path that the Duckiebot must follow starting at the bottom left

To make the Duckiebot follow the D-shaped path, we had to calculate the distance and angle the bot must travel to reach each point on the path. We then used the odometry equations to move the bot to each point.

The logic for the straight line and 90 degree on the spot rotation parts of this task were already handled in Part 2.

We now had to handle the logic for the two curved edges of the path

Our calculations are derived from equations (1) and (2) yet again. The angle covered is 90 degrees, but the right and left wheels also cover a finite distance. The right wheel covers a smaller distance than the left wheel due to the radius of its arc length being lesser than the left wheel by the length of the WHEELBASE

However, both wheels cover their respective distances in the same time. This means that the ratio of their velocities equals the ratio of their distances covered

Δt_left = Δt_right --> d_left / v_left = d_right / v_right --> v_left = (d_left / d_right) * v_right

d_left and d_right are calculated using the formula d = r * θ where r is the arc radius and θ is π/2 radians

We assume a reasonable value for v_right and calculate the corresponding value for v_left. Using these values, we apply the same logic as the rotation task to make the bot move in the path of the curved edge

3.2. Node 2: State Management and LED Service

We created an LED service that changes the color of the LED lights on the Duckiebot based on its state

Node 1 would publish the current state of the robot to a custom topic `/led_state`

The LED service node (Node 2) would subscribe to the aforementioned topic and based on the state publish a message to the `/led_emitter_node/led_pattern` topic to set the color of the Duckiebot's LED lights

The LED State Mappings are as follows:

- State 1: Stop - Robot is stationary for 5 seconds --> LED Lights are RED
- State 2: Trace "D" Path - Robot is moving along the D trajectory --> LED Lights are GREEN
- State 3: Return to Start - Robot has reached the starting position and is orienting itself. It waits for 5 seconds at starting position --> LED Lights are BLUE

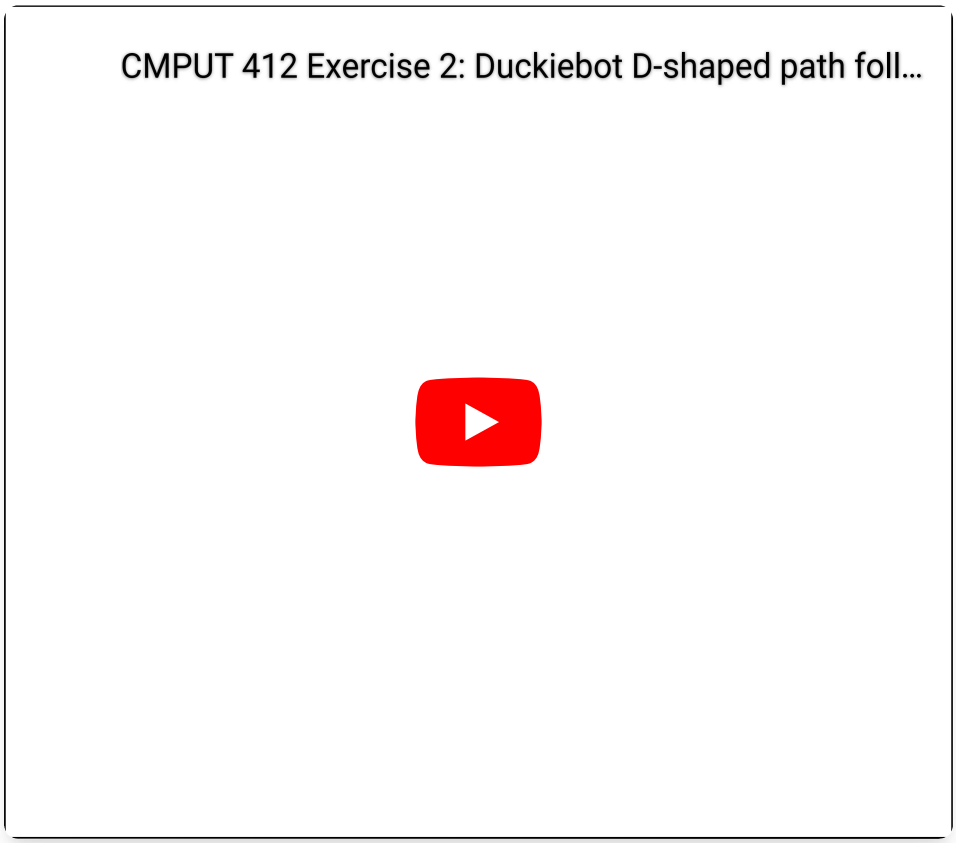


Fig 3.2: csc22926 travels autonomously along a D shaped path with different LED light colors for different states

2.4. Plotting Trajectory

We also plotted the trajectory of the Duckiebot in the world frame as it followed the D-shaped path

Fig 3.3: Trajectory of the D-shaped path according to csc22926(not necessarily the same as actual path travelled).

* Note that the start point and the point where the actual motion starts is different because the x and y values are calculated based on timestamps. Since the bot was stationary for the first 5 seconds, the timestamp values increased leading to a jump in position when it actually starts moving. Also axes are not actually in meters, that's a typo

Time elapsed: 47.54 secs

Is our duckiebot able to track what you planned according to your plan? If not, then why?

The Duckiebot has a Will of its own

- Our Duckiebot tracks a a reasonable D-shape based on the assignment specifications and the given surface. However, it is NOT actually tracking the same path that was planned for it. We can see that the trajectory plot, which is based on our assigned values, calculations and algorithms, traces a considerably different path than what we see in the actual video.
- There are several reasons. The major reason is the compounding of deviations (discussed in sections 2.2 and 2.3) during the duckiebot's journey. As the errors due to factors like wheel drift, slipping, sliding, callibration, etc compound, the duckiebot would veer more and more off track.
- So, even if our calculations are perfect for the given fixed trajectory, it is highly unlikely that the Duckiebot would follow the same D path
- In order to make the Duckiebot follow a reasonable D path, we accounted for potential errors by reducing the amount of rotation at hard corners, reducing the arc length it needs to travel at curved corners, giving one of its wheels a higher velocity for one subregion of the trajectory, etc
- This is why we can see that the trajectory plot does not plot an ideal D, because our algorithm itself is introducing intentional errors to combat the real life deviations

Part 4. Conclusion and Challenges

4.1. Conclusion

In this exercise, we explored the fundamentals of ROS and developed ROS-based software for DuckieTown using a Duckiebot. We performed some basic ROS operations including some basic OpenCV image processing.
The main task involved utilizing the Duckiebot's odometry (differential drive) and applying kinematic equations to control its movement along predefined trajectories and analyzing its motion.
We plotted the trajectory of the Duckiebot in the world frame based on the odometry information and analyzed the differences between the actual and expected paths. Additionally, we created an LED service to control the LED lights on the Duckiebot depending on its state during its motion.

4.2. Challenges

This exercise had quite a few challenges, mostly with finetuning our calculations to make the duckiebot actually move in D-shaped path
Accounting for the error in one region of the trajectory would introduce new errors in other parts of the trajectory
We spent many, many iterations running our program with slight modifications to eventually get the motion of the Duckiebot that we see in the video.

Another challenge was plotting the trajectory using the /wheel_cmd velocity data which are actually throttle values and not velocity in m/s which our kinematics equations expect
It took us a while to figure out that this was the case and we eventually settled on adding a scaling factor to the received velocity values to get an accuract trajectory plot

4.3. Reflection

Overall, while we encountered significant challenges, this lab was very rewarding. We learned a lot about ROS in a single exercise and the satisfaction of finally getting the duckiebot to start following reasonable D-shaped paths was second to none.
It was also a great exercise in understanding the importance of odometry and kinematics in controlling the motion of a robot while understanding

the complexities of deviations when relying on these metrics
Finally, the LED service was a fun addition to the project and a great way to visualize the state of the robot
I look forward to our future exercises where we leverage more of Duckiebot's sensors and apply more complex algorithms to control its motion!

Part 5. References and Acknowledgments

The following resources were used in the completion of this exercise:

- ROS Wiki for understanding ROS core concepts: https://wiki.ros.org/ROS/Tutorials
- Duckietown Documentation for developing in ROS: https://docs.duckietown.com/daffy/devmanual-software/beginner/ros/index.html
- ChatGPT helped with further strengthening ROS concepts and also providing an understanding of common ros commands like rostopic list, rosparam, rosnode, etc
- ChatGPT also helped provide the **animation code** for the trajectory plot (The actual trajectory equations and related code were developed by the main collaborators for this exercise)
- Kinematics equations were based on the following references: https://docs-old.duckietown.org/daffy/duckietown-robotics-development/out/odometry_modeling.html and the CMPUT 412 January 16, 2025 Lecture Slides on Differential Drive Kinematics

We would like to acknowledge the LI Adam Parker and the TAs Dikshant, Jasper and Monta for their assistance with explaining odometry concepts and possible causes for deviations, along with helping in understanding what commands to use for recording data into bag files and providing other ROS related tips.